



PROJET EN INFORMATIQUE I
INF34515 (T2)

UNIVERSITÉ DU QUÉBEC À RIMOUSKI

DÉPARTEMENT DE MATHÉMATIQUES, D'INFORMATIQUE ET DE GÉNIE

RAPPORT

**Développement d'une application mobile de suivi de
randonnée**

ÉTUDIANTE :
Nacera Sahmi

PROFESSEUR :
Yacine Yaddaden, Ph. D.

Date : 18 août 2025

Table des matières

1	Introduction	3
2	Objectif	3
3	Description du système créé	3
4	Structure des vues de l'application	4
5	Fonctionnement du système	4
5.1	Suivi GPS	4
5.2	Authentification	5
5.3	Gestion des données	5
5.4	Persistance des données	5
6	Architecture et modèle de données	6
6.1	Modèle de données Firestore	6
6.2	Sécurité et règles Firebase	6
6.2.1	Objectifs et principes	6
6.2.2	Règles Firestore	6
6.2.3	Règles Firebase Storage	8
6.3	Stockage des images (Firestore Storage)	8
6.4	Configuration des clients OAuth (Google)	8
7	Difficultés rencontrées	9
7.1	Problème de précision dans le calcul des distances	9
7.2	Problèmes d'intégration de l'authentification Google	9
7.2.1	Changement de méthode et nouvelles contraintes	10
7.2.2	Impact sur les tests	10
8	Notions apprises	12
9	Perspectives	12
10	Conclusion	12

1 Introduction

Ce rapport présente une application mobile de suivi de randonnée développée avec **React Native** et **Expo**. Elle permet aux utilisateurs de profiter d'une expérience simple et intuitive pour enregistrer et consulter leurs activités en plein air.

2 Objectif

L'objectif principal est de fournir un outil permettant de :

- Suivre en temps réel le parcours grâce au **GPS**.
- Enregistrer des **photos géolocalisées**.
- Consulter l'historique complet des randonnées.
- Gérer un **profil utilisateur** personnalisé.

L'application intègre **Firebase** pour l'authentification et la persistance des données, ainsi que **Google Maps Geocoding** pour enrichir les informations de localisation.

3 Description du système créé

L'application mobile de suivi de randonnée offre plusieurs fonctionnalités permettant à l'utilisateur de gérer l'ensemble de son expérience :

- **Connexion et inscription** : authentification sécurisée via **Firebase Authentication**, avec prise en charge de l'email/mot de passe et de la connexion Google.
- **Suivi en temps réel** : enregistrement du parcours grâce au **GPS**, affichage sur une carte avec **Google Maps**, calcul de la distance et du temps écoulé.
- **Prise de photos géolocalisées** : capture de photos pendant la randonnée avec localisation et adresse obtenues via **Google Maps Geocoding**.
- **Consultation de l'historique** : accès à la liste complète des randonnées enregistrées, avec détails du trajet, distances, durées et photos associées.
- **Gestion du profil utilisateur** : modification des informations personnelles (nom, prénom, date de naissance), consultation des données enregistrées et déconnexion.

4 Structure des vues de l'application

L'application est organisée en plusieurs vues principales, chacune correspondant à une fonctionnalité clé :

- **Accueil** : écran d'entrée de l'application permettant d'accéder à la connexion ou à l'inscription.
- **Connexion** : interface d'authentification avec email/mot de passe ou connexion via Google.
- **Inscription** : formulaire de création de compte avec informations personnelles.
- **Home** : page d'accueil après connexion, affichant un message de bienvenue et l'accès au suivi.
- **Suivi** : suivi GPS en temps réel avec affichage de la carte, distance parcourue, temps écoulé et possibilité de prendre des photos géolocalisées.
- **Historique** : liste des randonnées passées avec date, distance et durée.
- **Détails randonnée** : affichage détaillé d'une randonnée avec tracé sur carte et photos associées.
- **Mes Photos** : galerie des photos prises lors des randonnées, avec possibilité de suppression.
- **Profil** : page de gestion des informations personnelles de l'utilisateur.
- **PlusMenu** : menu supplémentaire donnant accès au profil et à la déconnexion.

5 Fonctionnement du système

Le fonctionnement de l'application repose sur plusieurs modules interconnectés assurant la gestion du suivi, de l'authentification et de la persistance des données.

5.1 Suivi GPS

Le suivi est réalisé à l'aide du module **Expo Location**, qui permet de récupérer en temps réel la position de l'utilisateur. Les coordonnées GPS sont enregistrées dans le store global via **Redux Toolkit** et affichées sur une carte Google Maps intégrée grâce à **React Native Maps** avec le fournisseur **PROVIDER_GOOGLE**. L'utilisateur peut également prendre des **photos géolocalisées** pendant la randonnée, enrichies par l'adresse obtenue via l'API **Google Maps Geocoding**. À la fin de la session, le parcours et les photos sont sauvegardés dans **Firebase Firestore**.

5.2 Authentification

L'application utilise **Firestore Authentication** pour sécuriser l'accès. Deux modes de connexion sont disponibles : par **email/mot de passe** ou via **Google OAuth**. Les informations de l'utilisateur connecté sont stockées dans Redux pour être accessibles dans toutes les vues.

5.3 Gestion des données

Les informations de randonnée (parcours, distance, durée, photos « en session ») ainsi que les données de profil sont centralisées côté client avec **Redux Toolkit**. Plusieurs *slices* (*userSlice*, *randoSlice*, *photoSlice*, *historiqueSlice*) structurent l'état pour simplifier l'affichage et les mises à jour en temps réel.

5.4 Persistance des données

La persistance est assurée par deux services complémentaires de Firebase :

- **Cloud Firestore** : stocke les *métadonnées* (profil, points GPS, sessions, et pour chaque photo : latitude/longitude, adresse, timestamp, url).
- **Firestore Storage** : stocke le *fichier image* lui-même. Après upload, l'URL sécurisée est enregistrée dans Firestore.

Cette séparation (métadonnées dans Firestore, médias dans Storage) optimise les performances et les coûts, et facilite la synchronisation multi-appareils. Les règles de sécurité restreignent l'accès aux documents et fichiers au seul utilisateur authentifié .

6 Architecture et modèle de données

6.1 Modèle de données Firestore

Chemin	Clé (ID)	Champs (type) — remarques
utilisateurs/{uid}	uid	nom (string), prenom (string), dateNaissance (string yyyy-MM-dd), email (string), genre (string)
suivis/{uid}/points/{pointId}	pointId	latitude (number), longitude (number), timestamp (string ISO)
suivis/{uid}/sessions/{sessionId}	sessionId	startTime (string ISO), endTime (string ISO), distance (number, m), duration (number, s), route (liste de {latitude, longitude}), photos (liste d'objets {url (https://), latitude, longitude, adresse (string), timestamp (ISO)})
photos/{uid}/items/{photoId}	photoId	url (https://), latitude (number), longitude (number), adresse (string), timestamp (string ISO)

TABLE 1 – Modèle de données Firestore — collections, documents et champs

6.2 Sécurité et règles Firebase

6.2.1 Objectifs et principes

Les règles Firebase protègent les données des utilisateurs, garantissent l'accès propriétaire (par *UID*) et valident le format des champs écrits par le client (dates, coordonnées, types, etc.). Nous appliquons le principe du *moindre privilège* : chaque utilisateur ne lit/écrit que ses propres documents et fichiers.

6.2.2 Règles Firestore

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function signedIn() { return request.auth != null; }
    function isOwner(userId) { return signedIn() && request.auth.uid == userId; }
    function isISODate(s) { return s is string && s.matches('^\\d{4}-\\d{2}-\\d{2}T.*Z$'); }
    function validLatLng(lat, lng) {
```

```
    return (lat is number && lat >= -90 && lat <= 90) &&
           (lng is number && lng >= -180 && lng <= 180);
}

// Profils
match /utilisateurs/{userId} {
  allow read: if isOwner(userId);
  allow create, update: if isOwner(userId);
  allow delete: if isOwner(userId);
}

// Points GPS temps rel
match /suivis/{userId}/points/{pointId} {
  allow read: if isOwner(userId);
  allow create: if isOwner(userId)
    && validLatLng(request.resource.data.latitude, request.resource.data.longitude)
    && isISODate(request.resource.data.timestamp);
  allow update, delete: if isOwner(userId);
}

// Sessions compltes
match /suivis/{userId}/sessions/{sessionId} {
  allow read: if isOwner(userId);
  allow create: if isOwner(userId)
    && (request.resource.data.distance is number && request.resource.data.distance >= 0)
    && (request.resource.data.duration is number && request.resource.data.duration >= 0)
    && isISODate(request.resource.data.startTime)
    && isISODate(request.resource.data.endTime)
    && request.resource.data.route is list;
  allow update, delete: if isOwner(userId);
}

match /photos/{userId}/items/{photoId} {
  allow read: if isOwner(userId);
  allow create, update, delete: if isOwner(userId);
}
}
```

Extrait de code 1 – Règles Cloud Firestore (code utilisé)

6.2.3 Règles Firebase Storage

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /photos/{uid}/{allPaths=**} {
      // LECTURE : juste lauth/UID
      allow read: if request.auth != null && request.auth.uid == uid;

      // ECRITURE : contraintes de taille/type
      allow write: if request.auth != null
        && request.auth.uid == uid
        && request.resource.size < 10 * 1024 * 1024
        && request.resource.contentType.matches('image/.*');
    }
  }
}
```

Extrait de code 2 – Règles Firebase Storage

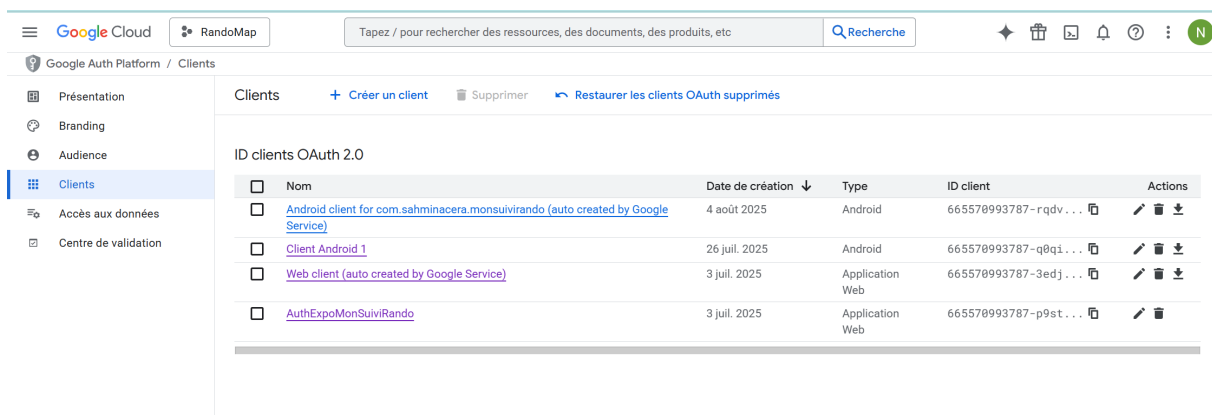
6.3 Stockage des images (Firebase Storage)

Les photos sont stockées dans **Firebase Storage** sous le chemin `photos/{uid}/{timestamp}.jpg`. Après upload, l'URL (`https://`) est obtenue via `getDownloadURL` et les métadonnées sont enregistrées dans Firestore (`photos/{uid}/items`) : url, latitude, longitude, adresse, timestamp. Côté sécurité, les règles limitent l'accès au propriétaire (uid), n'acceptent que `image/*` et une taille < 10 Mo (voir Extrait de code 2).

6.4 Configuration des clients OAuth (Google)

La figure 1 montre les clients OAuth configurés pour le projet. On distingue **deux familles** : (i) des *clients Web* utilisés pour les tests via **Expo/AuthSession** et (ii) des *clients Android* utilisés par l'app en production via **Google Sign-In** natif.

En pratique : dans l'application *compilée et installée* (production), seule l'authentification via le **client OAuth Android** (Google Sign-In natif) est utilisée. Les **clients Web** ne servent qu'aux essais avec **Expo/AuthSession** et ne sont pas requis dans l'application livrée.



☐	Nom	Date de création ↓	Type	ID client	Actions
☐	Android client for com.sahminacera.monsuivirando (auto created by Google Service)	4 août 2025	Android	665570993787-rqdv...	
☐	Client Android 1	26 juil. 2025	Android	665570993787-q8qi...	
☐	Web client (auto created by Google Service)	3 juil. 2025	Application Web	665570993787-3edj...	
☐	AuthExpoMonSuiviRando	3 juil. 2025	Application Web	665570993787-p9st...	

FIGURE 1 – Clients OAuth visibles dans Google Cloud

7 Difficultés rencontrées

Le développement de l'application a présenté plusieurs défis techniques, principalement liés au calcul précis des distances GPS et à l'intégration de la connexion Google.

7.1 Problème de précision dans le calcul des distances

Lors des premières versions, la distance parcourue était calculée avec la formule **Haversine**. Cette méthode ne tenait pas compte de certaines variations de trajectoire ni de la précision instantanée du GPS. Résultat : les distances enregistrées étaient souvent imprécises, parfois largement surestimées ou sous-estimées.

Pour améliorer ce point, la bibliothèque **Geolib** a été intégrée, utilisant la fonction `getPreciseDistance()`. Cette méthode permet de calculer la distance réelle entre chaque point GPS consécutif du parcours avec une meilleure précision. Cependant, même avec cette amélioration, la précision finale reste influencée par la qualité du signal GPS et les éventuelles pertes de position.

7.2 Problèmes d'intégration de l'authentification Google

La mise en place de la connexion Google s'est avérée plus complexe que prévu. Dans un premier temps, l'authentification a été développée avec **Expo AuthSession**. Sur iPhone via **Expo Go**, la page Google s'ouvrait correctement, mais la finalisation de l'authentification échouait avec le message :

“Something went wrong trying to finish signing in. Please close this screen to go back to the app.”

Cette erreur provient de la manière dont `AuthSession` gère les flux OAuth sur iOS dans **Expo Go**. Le processus repose sur une redirection via le proxy `https://auth.expo.io/@owner/slug`, qui sert d'intermédiaire entre le fournisseur d'authentification et

l'application. Or, Google applique désormais des règles de sécurité plus strictes pour les redirections OAuth, notamment sur iOS, ce qui entraîne parfois le rejet de la redirection finale lorsque celle-ci passe par le proxy d'Expo. Ainsi, bien que la page de connexion Google s'ouvre et que l'utilisateur puisse saisir ses identifiants, la validation finale échoue et génère le message d'erreur visible en figure 2. Pour contourner cette limitation, il est recommandé d'utiliser une bibliothèque native telle que `@react-native-google-signin/google-signin`, qui ne dépend pas du proxy et communique directement avec les API natives de la plateforme.

7.2.1 Changement de méthode et nouvelles contraintes

Une seconde approche a été mise en œuvre avec la bibliothèque `@react-native-google-signin/google-signin`, recommandée pour Google Sign-In car elle expose les API natives iOS/Android et retourne un `id_token` compatible Firebase. Cependant, cette bibliothèque inclut du **code natif** et ne peut pas être utilisée dans **Expo Go**. Elle impose donc un *build* personnalisé de l'application :

- **Android** : compilation et tests concluants (connexion Google fonctionnelle, voir figure 3) après génération d'un build de développement.
- **iOS** : deux options existent, aucune n'était disponible dans le cadre du projet au moment des tests :
 1. Compilation locale avec **Xcode** sur Mac et **Apple ID gratuit** (installation USB sur l'iPhone, signature valable 7 jours).
 2. Build cloud avec **EAS Build** et distribution **TestFlight**, qui requiert un **compte Apple Developer payant**.

7.2.2 Impact sur les tests

Les autres fonctionnalités (interfaces, navigation, etc.) ont été testées sur iPhone via **Expo Go**. En revanche, la **connexion Google** reste bloquée sur cet environnement, faute d'inclure le code natif requis. Des tests Android ont confirmé le bon fonctionnement de Google Sign-In, mais l'émulation Android est moins adaptée pour évaluer la partie **GPS**, qui bénéficie d'essais en conditions réelles (déplacements physiques).



FIGURE 2 – Erreur lors de la connexion Google sur iPhone avec Expo AuthSession (Expo Go).

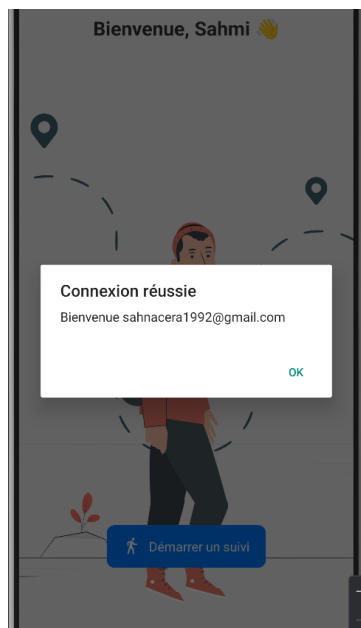


FIGURE 3 – Connexion Google réussie sur Android avec @react-native-google-signin/google-signin.

8 Notions apprises

La réalisation de ce projet a permis d'acquérir et de consolider plusieurs compétences techniques :

- **Redux Toolkit** : mise en place d'une gestion centralisée de l'état de l'application avec `createSlice` et `configureStore`, organisation en *slices* spécialisées pour les différentes fonctionnalités.
- **Google Maps API** : utilisation de l'API Geocoding pour convertir des coordonnées GPS en adresses lisibles, et affichage des parcours et marqueurs directement sur Google Maps.
- **Expo** : configuration et gestion du projet, utilisation d'outils et bibliothèques compatibles pour faciliter le développement et le déploiement multi-plateforme.
- **Firebase** : mise en œuvre de **Firebase Authentication** pour l'inscription et la connexion des utilisateurs, et utilisation de **Cloud Firestore** pour le stockage et la synchronisation des données.

9 Perspectives

Des améliorations futures pourraient inclure :

- L'optimisation des calculs de distance
- L'ajout de statistiques avancées (dénivelé, vitesse moyenne, etc.)
- L'intégration d'un mode hors ligne pour les zones sans réseau
- Le partage de parcours avec des amis ou via les réseaux sociaux
- La détection automatique de points d'intérêt
- L'envoi de notifications contextuelles (météo, alertes de sécurité)
- Le déploiement complet sur l'App Store et Google Play

Ces perspectives ouvrent la voie à de nouvelles fonctionnalités qui renforceraient l'intérêt et la polyvalence de **iHike** pour les randonneurs.

10 Conclusion

L'application **iHike** a permis de démontrer la faisabilité d'une solution mobile intégrée pour les randonneurs. Grâce à une interface intuitive, un suivi GPS fiable et la sauvegarde des photos géolocalisées, elle améliore significativement l'expérience utilisateur.

Les tests réalisés sur différents appareils ont confirmé la stabilité, la précision et la pertinence des fonctionnalités proposées. Le projet a ainsi répondu à ses objectifs en centralisant les outils essentiels dans une seule plateforme accessible.

Références

- [1] Expo. Expo documentation. <https://docs.expo.dev>. pages
- [2] Google Firebase. Firebase documentation. <https://firebase.google.com/docs>. pages
- [3] Google Maps Platform. Geocoding api documentation. <https://developers.google.com/maps/documentation/geocoding>. pages